

Universidade Federal de Alfenas – UNIFAL-MG

Instituto de Ciências Exatas – ICEx

Bacharelado em Ciência da Computação

Implementação do Protocolo Go-Back-N em Java via UDP

Relatório Técnico – Trabalho Final

Disciplina de Redes de Computadores

Autores: Pedro Finochio
Thiago Martins

Professor: Flavio Barbieri Gonzaga

Alfenas – MG

Julho de 2026

Sumário

1	Visão Geral da Solução	2
2	Fluxo Geral da Execução	2
3	Arquitetura do Projeto	4
3.1	Divisão Geral	4
3.2	Diagrama de Arquitetura	4
4	Organização dos Pacotes	5
5	Principais Classes	6
5.1	Pacote <code>common</code>	7
5.2	Pacote <code>sender</code>	7
5.3	Pacote <code>receiver</code>	8
6	Formato dos Pacotes	8
7	Negociação Inicial da Sessão	9
8	Implementação do Emissor	9
9	Implementação do Receptor	9
10	Máquinas de Estados	10
11	Decisões de Projeto	10
11.1	Serialização própria	10
11.2	Separação entre infraestrutura e protocolo	10
11.3	Temporizador único	11
11.4	Handshake independente	11
11.5	Organização modular	11
12	Dificuldades Encontradas	11
13	Estratégia de Testes	12
13.1	Ambiente de Testes	12
14	Resultados da Execução	12
14.1	Saída do programa durante a execução	13
15	Conclusão	15
16	Referências	15

1 Visão Geral da Solução

A solução desenvolvida consiste em uma aplicação distribuída composta por dois programas independentes escritos em Java: um módulo Emissor (**Sender**) e um módulo Receptor (**Receiver**). Ambos os módulos comunicam-se exclusivamente por meio de sockets UDP (`DatagramSocket` e `DatagramPacket`), implementando na camada de aplicação todos os mecanismos necessários para fornecer confiabilidade durante a transferência.

Inicialmente, o Emissor realiza um processo de negociação da sessão por meio de um pacote **HANDSHAKE** contendo informações como tamanho do arquivo, tamanho da janela, probabilidade de perda e caminho absoluto do arquivo de destino. Após o recebimento dessas informações, o Receptor inicializa sua sessão de recepção e passa a aceitar segmentos de dados.

O arquivo é dividido em blocos de até 1024 bytes, numerados sequencialmente e enviados conforme permitido pela janela deslizante configurada. Durante toda a execução, confirmações cumulativas permitem o avanço da janela de transmissão, enquanto eventos de *timeout* provocam retransmissões automáticas sempre que necessário.

Ao final da transferência, um pacote **FIN** encerra a comunicação, sendo apresentadas estatísticas referentes ao número de pacotes enviados, retransmitidos, perdas simuladas e *throughput* observado durante a execução.

Nota sobre progresso em tempo real: o enunciado sugere a exibição de progresso durante a transmissão (Seção 3.3, R6). Na classe efetivamente utilizada pelo bootstrap (**Sender** → `SenderFSM`, pacote `sender.protocol`), o método `run()` é bloqueante e não expõe callback nem acesso à `SenderStatistics` durante a execução; por isso, o que é exibido são as estatísticas finais consolidadas após o término da transmissão, e não uma barra de progresso incremental. O código-fonte contém uma segunda implementação da máquina de estados do Emissor, `GbnSenderFsm` (pacote `sender.fsm`), que sim implementa impressão de progresso em tempo real a cada `ACK` recebido – mas essa classe não é referenciada por `Sender.java` e, portanto, não está em uso na versão atual do sistema.

2 Fluxo Geral da Execução

A Figura 1 apresenta o fluxo de execução do Emissor, reconstruído diretamente a partir da lógica implementada em `SenderFSM` (pacote `sender.protocol`, efetivamente invocada por `Sender.java`). O fluxo cobre as três fases sequenciais da FSM: **HANDSHAKE**, transmissão de **DATA** sob janela deslizante, e encerramento com **FIN**.

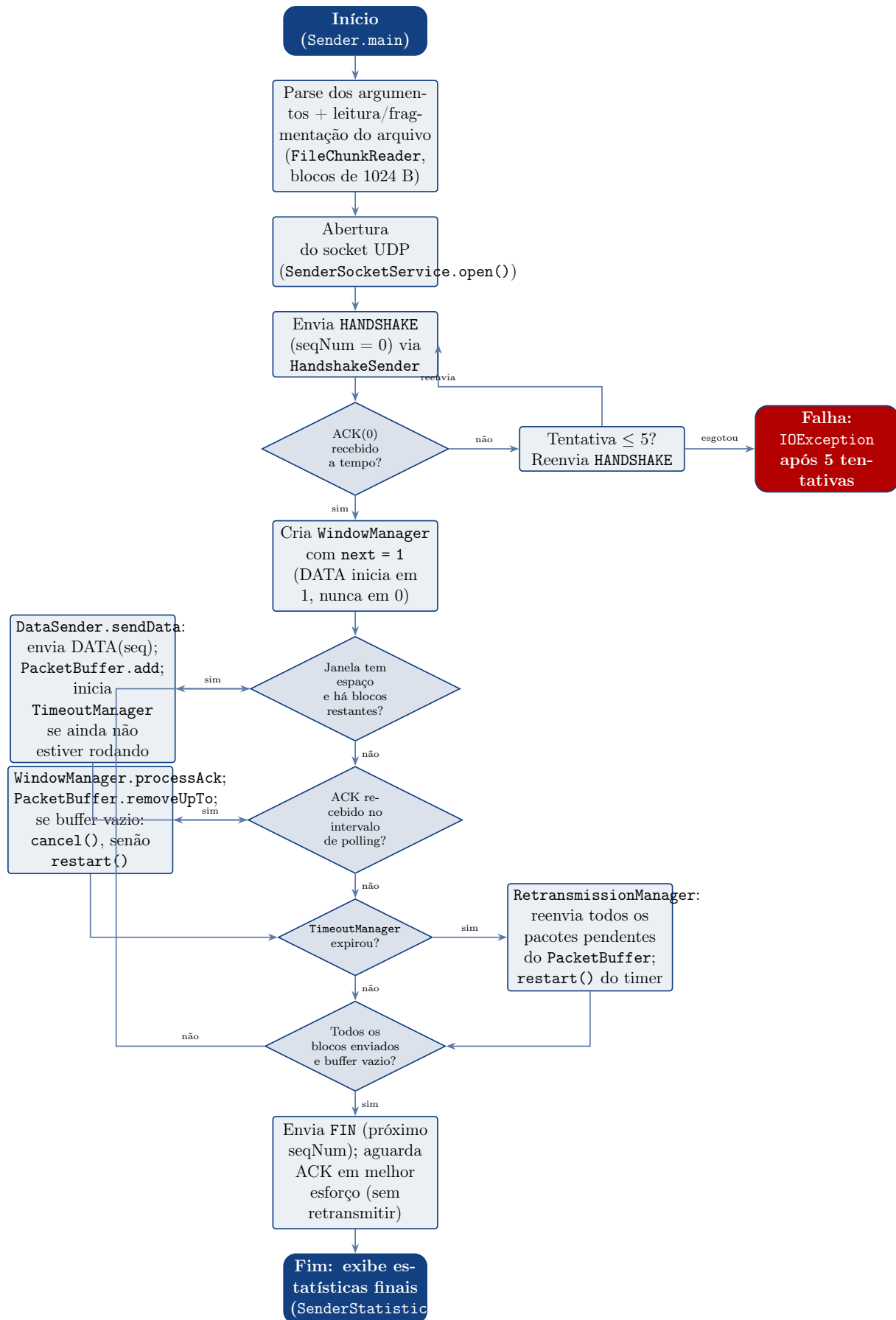


Figura 1: Fluxo de execução do Emissor (SenderFSM), da inicialização ao encerramento.

O laço central (envio sob janela → tentativa de recepção de ACK → verificação de *time-out*) repete-se até que todos os blocos do arquivo tenham sido enviados e o `PacketBuffer` esteja vazio – ou seja, até que o último ACK cumulativo confirme o último bloco. Somente então o Emissor avança para o envio do pacote FIN.

3 Arquitetura do Projeto

A implementação foi desenvolvida utilizando Java 21 e organizada como um único projeto Maven (`gbn-udp`), sem dependências externas além do JDK padrão, separando claramente os componentes compartilhados daqueles específicos do Emissor e do Receptor.

Toda a arquitetura foi planejada para manter baixo acoplamento entre as classes, facilitando manutenção, testes e futuras extensões do protocolo. Cada componente possui uma responsabilidade bem definida, evitando concentração excessiva de lógica em uma única classe. A divisão em pacotes também permite que futuras implementações de outros protocolos confiáveis, como *Selective Repeat*, reutilizem grande parte da infraestrutura já construída.

3.1 Divisão Geral

O projeto foi dividido em três pacotes principais:

- **common**: reúne todas as estruturas compartilhadas pelos dois módulos, incluindo representação dos pacotes, serialização e parâmetros da sessão;
- **sender**: contém toda a lógica relacionada ao envio confiável dos dados;
- **receiver**: concentra a lógica responsável pelo recebimento, validação e reconstrução do arquivo.

Essa separação reduz duplicação de código e torna explícita a responsabilidade de cada parte do sistema.

3.2 Diagrama de Arquitetura

A Figura 2 mostra os componentes efetivamente utilizados em cada lado da comunicação, com as classes nomeadas exatamente como aparecem no código-fonte. As setas horizontais tracejadas representam a única forma de comunicação entre os dois lados: datagramas UDP trocados entre `SenderSocketService` e o `DatagramSocket` aberto por `Receiver`.

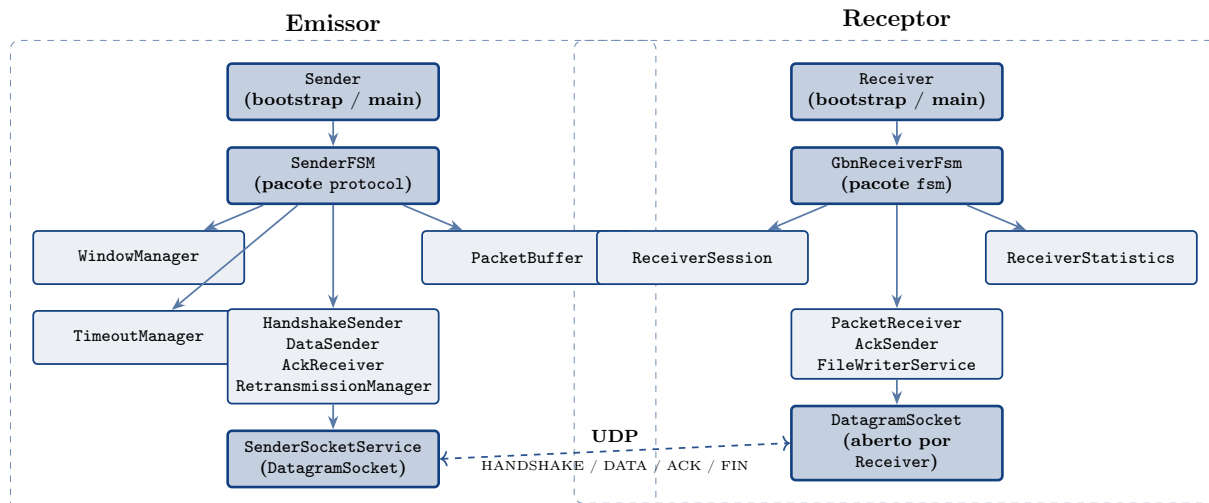


Figura 2: Componentes do Emissor e do Receptor efetivamente utilizados em tempo de execução, e a comunicação via UDP entre ambos.

O Emissor e o Receptor executam como processos independentes e comunicam-se exclusivamente através de datagramas UDP. Toda a lógica de confiabilidade é implementada na camada de aplicação, sem utilizar qualquer funcionalidade própria do protocolo TCP. No lado do Emissor, **SenderFSM** (pacote `sender.protocol`) é a classe efetivamente orquestrada por `Sender.java`; no lado do Receptor, **GbnReceiverFsm** (pacote `receiver.fsm`) cumpre o papel equivalente, sendo instanciada e executada por `Receiver.java` através do método `executar(...)`.

4 Organização dos Pacotes

A estrutura de diretórios do código-fonte é apresentada a seguir, refletindo a organização dos pacotes Java do projeto.

```

1 src/main/java/br/unifal/redes/
2 |-- Main.java                (classe vazia, sem uso)
3 |
4 |-- common/                  Classes compartilhadas entre Emissor e Receptor
5 |   |-- Packet.java
6 |   |-- PacketType.java
7 |   |-- PacketCodec.java
8 |   |-- PacketSerializer.java
9 |   '-- SessionParameters.java
10 |
11 |-- sender/                  Modulo Emissor
12 |   |-- Sender.java          (ponto de entrada / main)
13 |   |-- fsm/
14 |       '-- GbnSenderFsm.java (*) implementacao alternativa, nao referenciada
15 |   |-- io/
16 |       '-- FileChunkReader.java
17 |   |-- network/
18 |       |-- SenderSocketService.java
19 |       |-- WindowManager.java
20 |       |-- TimeoutManager.java
21 |       '-- PacketBuffer.java
22 |   |-- protocol/
23 |       |-- SenderFSM.java    (**) classe efetivamente usada por Sender.java
24 |       |-- HandshakeSender.java
25 |       |-- DataSender.java
26 |       |-- AckReceiver.java
27 |       '-- RetransmissionManager.java
28 |   |-- session/
29 |       '-- SenderSession.java (*) container de estado, nao referenciado
30 |   '-- statistics/
31 |       '-- SenderStatistics.java
32 |
33 '-- receiver/                Modulo Receptor
34 |   |-- Receiver.java         (ponto de entrada / main)
35 |   |-- fsm/
36 |       '-- GbnReceiverFsm.java (**) classe efetivamente usada por Receiver.java
37 |   |-- io/
38 |       '-- FileWriterService.java
39 |   |-- network/
40 |       |-- PacketReceiver.java
41 |       |-- AckSender.java
42 |       '-- IncomingPacket.java
43 |   |-- session/
44 |       '-- ReceiverSession.java
45 |   '-- statistics/
46 |       '-- ReceiverStatistics.java

```

Figura 3: Organização dos pacotes e classes do projeto, conforme o código-fonte compilado. (*) Classes presentes no repositório mas não referenciadas pelo fluxo de execução principal. (**) Classes efetivamente orquestradas pelos pontos de entrada `Sender.java` e `Receiver.java`.

Nota de consistência: o `README.md` do repositório descreve `SenderFSM` como pertencente ao pacote `sender.fsm`. A inspeção do código mostra que essa classe está, na verdade, em `sender.protocol`, e que o pacote `sender.fsm` contém uma implementação alternativa (`GbnSenderFsm`) não invocada por nenhum outro ponto do sistema. Este relatório segue a árvore de diretórios real do código-fonte, e não a descrição do `README.md`.

5 Principais Classes

Nesta seção são descritas as classes centrais do projeto e suas respectivas responsabilidades.

5.1 Pacote common

- **Packet**: representação imutável de um segmento do protocolo GBN, contendo tipo, número de sequência, número de confirmação e payload.
- **PacketType**: enumeração dos quatro tipos de pacote suportados pelo protocolo: DATA, ACK, HANDSHAKE e FIN.
- **PacketCodec**: responsável pela serialização e desserialização binária dos pacotes, utilizando `java.nio.ByteBuffer` em *big-endian*.
- **PacketSerializer**: responsável pela serialização dos parâmetros de sessão (**SessionParameters**) transmitidos no payload do pacote de *handshake*.
- **SessionParameters**: objeto de valor contendo os parâmetros negociados no *handshake* (tamanho do arquivo, tamanho da janela, probabilidade de perda e caminho de destino).

5.2 Pacote sender

- **Sender**: ponto de entrada (**main**) do módulo Emissor; realiza o *parsing* dos argumentos de linha de comando e orquestra a inicialização dos demais componentes.
- **SenderFSM**: implementa a máquina de estados finita do Emissor, incluindo a fase de *handshake*, o controle da janela deslizante e o tratamento de *timeout*.
- **FileChunkReader**: responsável pela leitura e fragmentação do arquivo de origem em segmentos de tamanho fixo.
- **SenderSocketService**: encapsula o `DatagramSocket` utilizado pelo Emissor para envio e recepção de datagramas.
- **WindowManager**: controla os ponteiros **base** e **nextseqnum** da janela deslizante de transmissão.
- **TimeoutManager**: gerencia o temporizador único associado ao pacote mais antigo não confirmado, com verificação por *polling*.
- **PacketBuffer**: mantém em memória os últimos N pacotes enviados, permitindo retransmissão sem necessidade de nova leitura em disco.
- **HandshakeSender**, **DataSender**, **AckReceiver**, **RetransmissionManager**: componentes responsáveis, respectivamente, pelo envio do pacote de *handshake*, envio de pacotes de dados, recepção de confirmações e retransmissão em rajada no evento de *timeout*.
- **SenderSession**: contêiner de estado da sessão de transmissão em andamento.
- **SenderStatistics**: contabiliza pacotes enviados, retransmitidos, ACKs recebidos e demais métricas exibidas ao final da transmissão.

5.3 Pacote receiver

- **Receiver**: ponto de entrada (**main**) do módulo Receptor; inicializa a porta UDP configurada e aguarda o *handshake* do Emissor.
- **GbnReceiverFsm**: implementa a máquina de estados do Receptor GBN, incluindo a política de aceitação somente de pacotes em ordem e a simulação de perda.
- **FileWriterService**: responsável pela escrita sequencial do arquivo recebido em disco, no caminho absoluto informado pelo Emissor.
- **PacketReceiver**: encapsula a recepção de datagramas UDP pelo Receptor.
- **AckSender**: responsável pelo envio de confirmações (ACK) ao Emissor.
- **IncomingPacket**: objeto de valor que associa um pacote recebido ao endereço de origem do remetente.
- **ReceiverSession**: mantém o estado da sessão de recepção, incluindo o ponteiro `expectedseqnum`.
- **ReceiverStatistics**: contabiliza pacotes recebidos, aceitos, descartados e ACKs enviados, exibidos ao final da recepção.

6 Formato dos Pacotes

Todos os datagramas transmitidos seguem um formato binário próprio definido para o projeto. Cada pacote possui um cabeçalho fixo de 11 bytes contendo informações necessárias ao funcionamento do protocolo, seguido por um campo de dados de até 1024 bytes cujo conteúdo depende do tipo da mensagem.

Tabela 1: Formato do cabeçalho dos pacotes.

Campo	Tamanho	Descrição
Tipo	1 byte	HANDSHAKE, DATA, ACK ou FIN
Número de sequência	4 bytes	Identificador do segmento
Número de ACK	4 bytes	Confirmação cumulativa
Tamanho do payload	2 bytes	Quantidade de bytes úteis
Payload	até 1024 bytes	Dados transportados

O uso de um formato único simplifica a serialização e desserialização dos datagramas, permitindo que diferentes tipos de mensagens reutilizem exatamente a mesma estrutura de pacote. Toda serialização foi implementada utilizando a classe `ByteBuffer`, em *big-endian*, garantindo representação binária consistente independentemente da arquitetura da máquina.

7 Negociação Inicial da Sessão

Antes do envio de qualquer segmento de dados é realizado um processo de negociação entre Emissor e Receptor. Essa etapa utiliza um pacote do tipo **HANDSHAKE**, cujo payload contém todas as informações necessárias para inicializar a transferência. São transmitidos:

- tamanho total do arquivo;
- tamanho da janela de transmissão;
- probabilidade de perda simulada;
- caminho absoluto onde o arquivo será salvo.

Após receber esse pacote, o Receptor inicializa sua sessão e permanece aguardando os segmentos de dados. Esse procedimento elimina a necessidade de configurações adicionais no Receptor e garante que toda a sessão seja parametrizada automaticamente pelo Emissor. O número de sequência 0 é reservado exclusivamente ao **HANDSHAKE** e ao seu **ACK**, evitando colisão com a numeração dos pacotes de dados, que se inicia em 1.

8 Implementação do Emissor

O Emissor é responsável por toda a lógica de transmissão confiável dos dados. Seu funcionamento pode ser dividido em cinco etapas principais:

1. leitura e validação dos parâmetros de linha de comando;
2. leitura do arquivo de origem;
3. envio do pacote **HANDSHAKE**;
4. transmissão dos segmentos utilizando janela deslizante;
5. encerramento da sessão por meio do pacote **FIN**.

Durante a transmissão são mantidas as duas variáveis fundamentais do protocolo Go-Back-N: **base**, correspondente ao primeiro pacote ainda não confirmado, e **nextSeqNum**, correspondente ao próximo número de sequência disponível. Enquanto existir espaço livre na janela, novos segmentos podem ser enviados imediatamente. Sempre que um **ACK** cumulativo é recebido, a janela desliza, permitindo novas transmissões. Caso o temporizador expire, todos os segmentos ainda pertencentes à janela são retransmitidos.

9 Implementação do Receptor

O Receptor permanece aguardando conexões em uma porta UDP configurável. Após receber o **HANDSHAKE**, são inicializadas todas as estruturas necessárias para reconstrução do arquivo. A implementação mantém apenas uma variável de estado, denominada **expectedSeqNum**, correspondente ao próximo segmento esperado.

Sempre que um pacote **DATA** é recebido, três situações podem ocorrer:

1. o pacote chega exatamente na ordem esperada;

2. o pacote chega fora de ordem;
3. o pacote, embora em ordem, é descartado pela simulação probabilística de perda.

Quando o segmento possui o número de sequência esperado e não é descartado pela simulação, seus dados são gravados imediatamente no arquivo de destino e um ACK cumulativo é enviado ao Emissor. Pacotes fora de ordem são descartados conforme especificação do protocolo Go-Back-N, sendo reenviado o último ACK válido. Nos cenários em que ocorre perda simulada, o pacote é simplesmente ignorado, não sendo enviado qualquer ACK, forçando naturalmente a retransmissão pelo Emissor após o *timeout*.

10 Máquinas de Estados

A implementação segue o modelo de máquinas de estados finitas apresentado por Kurose e Ross. No Emissor, os principais eventos considerados são:

- envio de novos segmentos;
- recepção de ACKs;
- expiração do temporizador.

No Receptor, os eventos relevantes são:

- chegada de um pacote HANDSHAKE;
- chegada de um segmento DATA;
- chegada de um pacote FIN.

Essa abordagem torna o comportamento do protocolo completamente determinístico e facilita tanto sua implementação quanto sua depuração.

11 Decisões de Projeto

Durante o desenvolvimento foram adotadas algumas decisões que contribuíram para simplificar a implementação e aumentar sua robustez.

11.1 Serialização própria

Foi desenvolvido um mecanismo próprio de serialização dos pacotes utilizando exclusivamente classes da biblioteca padrão do Java. Essa abordagem elimina dependências externas e produz datagramas compactos.

11.2 Separação entre infraestrutura e protocolo

As classes responsáveis pela comunicação UDP possuem apenas funções relacionadas ao envio e recebimento de datagramas. Toda a lógica do protocolo permanece isolada nas máquinas de estados. Essa separação facilita testes e reduz o acoplamento entre componentes.

11.3 Temporizador único

Seguindo a especificação do protocolo Go-Back-N, apenas o pacote mais antigo da janela possui temporizador associado. O temporizador foi implementado por *polling* (`TimeoutManager`, baseado em `System.nanoTime()`), em vez de *threads* dedicadas, reduzindo a complexidade da sincronização quando comparado a protocolos como o *Selective Repeat*.

11.4 Handshake independente

A negociação inicial foi implementada separadamente da transmissão dos dados. Isso permite que futuras informações de configuração sejam adicionadas ao protocolo sem modificar o funcionamento da janela deslizante.

11.5 Organização modular

Cada classe possui uma responsabilidade única, seguindo princípios de orientação a objetos. Essa decisão tornou a implementação significativamente mais organizada, facilitando tanto o desenvolvimento quanto a manutenção futura do sistema.

12 Dificuldades Encontradas

O desenvolvimento do protocolo Go-Back-N apresentou desafios tanto de natureza conceitual quanto de implementação.

A primeira dificuldade consistiu na definição de uma arquitetura suficientemente modular para evitar que a lógica do protocolo ficasse concentrada em poucas classes. Optou-se por separar claramente as responsabilidades entre comunicação UDP, serialização, manipulação de arquivos, gerenciamento da sessão e implementação da máquina de estados, permitindo que cada componente permanecesse responsável por apenas uma parte do sistema.

Outro desafio importante foi a implementação correta da máquina de estados do Emissor. O protocolo Go-Back-N exige o controle simultâneo dos ponteiros `base` e `nextSeqNum`, além da sincronização entre o envio de novos pacotes, o recebimento de ACKs cumulativos e a ocorrência de eventos de *timeout*. Pequenos erros nessa lógica podem causar retransmissões desnecessárias ou impedir o avanço da janela de transmissão.

A implementação do temporizador também exigiu atenção especial. Conforme especificado pelo protocolo, apenas o pacote mais antigo ainda não confirmado deve possuir temporização ativa, sendo necessário garantir que o temporizador fosse reiniciado sempre que a base da janela avançasse e cancelado quando todos os pacotes fossem confirmados.

Também houve preocupação com a definição do formato binário dos datagramas, sendo necessário estabelecer um layout fixo para todos os pacotes transmitidos, permitindo que Emissor e Receptor interpretassem corretamente os campos de controle independentemente do tipo da mensagem.

Finalmente, a simulação probabilística de perda exigiu cuidados para representar corretamente o comportamento esperado pelo protocolo. Os descartes são realizados exclusivamente no Receptor e apenas sobre pacotes recebidos em ordem, permitindo validar o mecanismo de retransmissão sem depender de perdas reais da rede.

13 Estratégia de Testes

Durante o desenvolvimento foram realizados diversos testes incrementais para validar isoladamente cada etapa do protocolo, incluindo o processo de *handshake*, a serialização e desserialização binária dos pacotes, o envio e recepção dos segmentos de dados, a geração e o processamento das confirmações (ACKs), o disparo de eventos de *timeout* e a reconstrução do arquivo no Receptor. Esses testes incrementais foram utilizados apenas para validar o funcionamento interno do sistema durante a implementação, não constituindo uma campanha experimental formal.

Este relatório documenta exclusivamente o teste oficial solicitado pelo professor, conforme especificado no enunciado do trabalho (Seção 6, "Entrega e Apresentação"). O teste consiste na transferência de um arquivo com tamanho superior a 1 MB, utilizando probabilidade de perda configurada em 10%, exatamente conforme exigido para a apresentação em laboratório.

13.1 Ambiente de Testes

O teste foi executado em ambiente *localhost* (endereço 127.0.0.1), com Emissor e Receptor rodando como processos independentes na mesma máquina e comunicando-se via sockets UDP, utilizando Java 21. O arquivo transmitido foi `tp_safety.pdf`, com tamanho de 1.074.655 bytes, dividido em blocos de 1024 bytes. Os parâmetros utilizados foram: tamanho da janela (N) igual a 4, probabilidade de perda configurada em 10% e temporizador de retransmissão (*timeout*) de 500 ms.

14 Resultados da Execução

Na execução oficial, o *handshake* entre Emissor e Receptor foi concluído com sucesso, com o Receptor criando a sessão de recepção a partir dos parâmetros recebidos (destino, tamanho da janela e probabilidade de perda). Em seguida, os dados do arquivo `tp_safety.pdf` foram transmitidos em blocos sob controle da janela deslizante. Durante a transmissão ocorreram retransmissões provocadas pelas perdas simuladas no Receptor, e o protocolo Go-Back-N respondeu a essas perdas retransmitindo os pacotes pendentes da janela após a expiração do temporizador. Ao final, a transferência foi concluída corretamente, com o arquivo integralmente recebido pelo Receptor.

A Tabela 2 resume os dados obtidos a partir dos logs de execução do Emissor e do Receptor.

Tabela 2: Resumo dos dados da execução oficial do protocolo Go-Back-N.

Configuração	
Arquivo	tp_safety.pdf
Tamanho	1.074.655 bytes
Janela	4
Perda configurada	10%
Timeout	500 ms
Emissor	
Pacotes enviados	1503
Retransmitidos	452
ACKs recebidos	1390
Timeouts	113
Bytes enviados (rede)	1.537.503
Throughput efetivo	17,14 KB/s
Taxa de retransmissão	30,07%
Receptor	
Pacotes recebidos	1502
Pacotes aceitos	1050
Pacotes descartados	452
ACKs enviados	1050
Taxa de perda efetiva	30,09%

Observa-se que a taxa de perda/retransmissão efetivamente observada (em torno de 30%) ficou superior aos 10% de probabilidade de perda configurada. Isso ocorre porque, no protocolo Go-Back-N, cada evento de *timeout* provoca a retransmissão de **todos** os pacotes ainda não confirmados dentro da janela, e não apenas do pacote efetivamente perdido. Dessa forma, uma única perda simulada pode gerar múltiplos pacotes retransmitidos, aumentando naturalmente o número total de pacotes enviados e observados em relação ao número de blocos originais do arquivo (1050).

14.1 Saída do programa durante a execução

A seguir são reproduzidos, na íntegra, os logs de saída gerados pelo Receptor e pelo Emissor durante a execução oficial.

```

1 [Receptor] Iniciando na porta 5000.
2
3 [Receptor] Infraestrutura pronta. Aguardando conexao do Emissor.
4
5 [FSM] 1 - Entrou em processarHandshake()
6
7 [FSM] 2 - Datagrama recebido
8
9 [FSM] 3 - Tipo do pacote: HANDSHAKE
10
11 [FSM] 4 - HANDSHAKE validado
12

```

```

13 [FSM] 5 - Remetente: 127.0.0.1:59365
14
15 [FSM] 6 - SessionParameters desserializados
16
17 [FSM] destino = receptor_tp_safety.pdf
18
19 [FSM] janela = 4
20
21 [FSM] perda = 0.1
22
23 [FSM] 7 - Arquivo aberto
24
25 [FSM] 8 - Sessao criada
26
27 [FSM] 9 - ACK(0) enviado
28
29 [FSM] 10 - expectedSequenceNumber = 1
30
31 [Receptor] Transferencia concluida.
32
33
34
35 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
36
37 X          ESTATISTICAS DA RECEPCAO          X
38
39 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
40
41 X Pacotes recebidos (total): 1502          X
42
43 X Pacotes aceitos:          1050          X
44
45 X Pacotes descartados:          452          X
46
47 X ACKs enviados:          1050          X
48
49 X Taxa de perda efetiva:          30,09%          X
50
51 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.

1 [Emissor] Arquivo de origem : C:\Projects\tpRedes\tpRedes\tp_safety.pdf
2 [Emissor] Destino : 127.0.0.1:5000 -> receptor_tp_safety.pdf
3 [Emissor] Tamanho da janela : 4
4 [Emissor] Prob. de perda : 10,00%
5 [Emissor] Timeout : 500 ms
6
7 [Emissor] Arquivo lido: 1.074.655 bytes em 1.050 bloco(s) de 1024 bytes
8
9 [Emissor] Socket aberto na porta local 59365.
10 [Emissor] Aguardando Receptor em 127.0.0.1:5000 ...
11
12
13 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
14 X          ESTATISTICAS DA TRANSMISSAO          X
15 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
16 X Pacotes enviados (total): 1503          X
17 X Pacotes retransmitidos: 452          X
18 X ACKs recebidos: 1390          X
19 X Timeouts de retransmissao: 113          X
20 X Bytes enviados (na rede): 1.537.503          X
21 X Taxa de retransmissao: 30,07%          X
22 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
23 X Duracao total: 1m 01s          X
24 X Throughput efetivo: 17,14 KB/s          X

```

25 XXX

15 Conclusão

O objetivo deste trabalho consistiu na implementação de uma camada de transporte confiável sobre o protocolo UDP utilizando o algoritmo Go-Back-N. Foram implementados os módulos Emissor e Receptor, o mecanismo de negociação inicial da sessão, o protocolo de transferência baseado em janela deslizante, o envio de confirmações cumulativas, a retransmissão após *timeout*, a simulação probabilística de perda de pacotes e a reconstrução do arquivo recebido.

A arquitetura modular adotada durante o desenvolvimento permitiu separar claramente as responsabilidades entre os diferentes componentes do sistema, facilitando tanto a implementação quanto a manutenção do código.

O teste oficial solicitado pelo professor foi realizado com sucesso: o arquivo `tp_safety.pdf`, com tamanho superior a 1 MB (1.074.655 bytes), foi transmitido do Emissor para o Receptor com probabilidade de perda configurada em 10%. Diante dos eventos de perda simulada, o protocolo Go-Back-N respondeu corretamente por meio das retransmissões da janela após expiração do temporizador, e o sistema concluiu a transferência do arquivo de forma consistente, validando o funcionamento integrado dos mecanismos de *handshake*, janela deslizante, confirmações cumulativas, temporizador e retransmissão implementados.

Como possível extensão futura, destaca-se a implementação do protocolo *Selective Repeat*, permitindo comparação experimental entre diferentes estratégias ARQ e análise do impacto de cada abordagem sobre o desempenho da comunicação.

16 Referências

- KUROSE, James F.; ROSS, Keith W. **Redes de Computadores: Uma Abordagem Top-Down**. 8. ed. São Paulo: Pearson, 2021. Capítulo 3, Seções 3.4 e 3.4.3.
- TANENBAUM, Andrew S.; WETHERALL, David. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011. Capítulo 3.
- ORACLE. **Java Platform, Standard Edition 21 API Specification**. Oracle Corporation.